



CYNA

Plan de tests de performance et de charge

Projet fil rouge — Plateforme Cyna

**Adrien CACHOUX · Romain CANER ·
Ethan MIRBEAU**

Version 1.0 — Bloc 3

Solutions de cybersécurité SaaS — SOC · EDR · XDR

Ce document décrit les **tests de performance et de montée en charge** du site web Cyna : objectifs, outillage, scénarios, procédure d'exécution et grille de résultats. Il répond à l'axe « Tests & performances » du Bloc 3, en complément des tests unitaires/fonctionnels du Cahier de recettes.

1. Objectifs et critères d'acceptation

Dérivés des KPI de la note de cadrage (§4.4) :

Critère	Seuil d'acceptation
Temps de réponse moyen (pages clés)	< 2 s
Temps de réponse P95	< 3 s
Taux d'erreurs sous charge nominale	< 1 %
Débit soutenu (VUs simultanés visés)	≥ 50 utilisateurs virtuels
Stabilité (endurance)	Aucune fuite mémoire sur 30 min

2. Outillage

Outil retenu : **k6** (Grafana k6) — léger, scriptable en JavaScript, adapté à un VPS modeste. Alternative acceptée : **Apache JMeter** (interface graphique) ou **ab** (Apache Bench) pour un test rapide.

```
# Installation k6 (Debian/Ubuntu)
sudo gpg -k && sudo apt-get install k6
# ou via Docker :
docker run --rm -i grafana/k6 run - < scripts/perf/charge.js
```

3. Scénarios de charge

3.1 Parcours testés

Scénario	Parcours	Poids
S1 — Navigation	Accueil → catégorie → fiche produit	50 %
S2 — Recherche	Recherche avec facettes	20 %
S3 — Tunnel de commande	Ajout panier → panier → connexion	20 %
S4 — Back-office	Connexion admin (TOTP) → liste produits	10 %

3.2 Profils de charge

Type de test	Objectif	Profil
Charge nominale	Valider les seuils en usage normal	Montée à 50 VUs, palier 5 min
Montée en charge (stress)	Trouver le point de rupture	Paliers 10 → 100 → 200 VUs
Pic (spike)	Résister à un afflux soudain	0 → 150 VUs en 30 s
Endurance (soak)	Détecter fuites / dégradation	30 VUs pendant 30 min

4. Script de référence k6

Fichier fourni : [scripts/perf/charge.js](#) .

```
import http from 'k6/http';
import { check, sleep, group } from 'k6';

const BASE = __ENV.BASE_URL || 'http://localhost:8080';

export const options = {
  scenarios: {
    charge_nominale: {
      executor: 'ramping-vus',
      startVUs: 0,
      stages: [
        { duration: '1m', target: 50 }, // montée
        { duration: '5m', target: 50 }, // palier
        { duration: '1m', target: 0 }, // descente
      ],
    },
  },
  thresholds: {
    http_req_duration: ['avg<2000', 'p(95)<3000'], // < 2 s moyen, < 3 s P95
    http_req_failed: ['rate<0.01'], // < 1 % d'erreurs
  },
};

export default function () {
  group('S1 - Navigation', () => {
    const home = http.get(`${BASE}/`);
    check(home, { 'accueil 200': (r) => r.status === 200 });
    sleep(1);
    const cat = http.get(`${BASE}/categories`);
    check(cat, { 'catégories 200': (r) => r.status === 200 });
    sleep(1);
  });

  group('S2 - Recherche', () => {
    const res = http.get(`${BASE}/recherche?q=soc`);
    check(res, { 'recherche 200': (r) => r.status === 200 });
    sleep(1);
  });
}
```

Adapter les chemins (`/categories` , `/recherche`) aux routes réelles de l'application. Utiliser un jeu de données de démonstration (`seed.sql`) sur un **environnement dédié** — ne jamais tirer de charge sur la production réelle sans fenêtre planifiée.

5. Procédure d'exécution

```
# 1. Démarrer un environnement isolé
cd Cyna && docker compose up -d --build

# 2. Lancer le test de charge nominale
BASE_URL=http://localhost:8080 k6 run scripts/perf/charge.js

# 3. Test de stress (surcharge des paliers via variable)
k6 run --stage 1m:100 --stage 2m:200 scripts/perf/charge.js

# 4. Récupérer le rapport résumé affiché par k6 en fin de run
```

6. Grille de résultats (à compléter après exécution)

Test	Date	VUs max	Req/s	Temps moyen	P95	Erreurs	Verdict
Charge nominale		50	—	— ms	— ms	— %	
Stress		200	—	— ms	— ms	— %	
Pic	🟢	150 🟡	—	🔴 — ms	— ms	— %	
Endurance		30	—	— ms	— ms	— %	

Légende : Conforme · Réserve · Non conforme · À exécuter

7. Analyse et pistes d'optimisation

Les leviers déjà en place ou activables si les seuils sont dépassés :

- **Index SQL** sur colonnes de tri/filtre (DCT §7.2) — guidés par le slow query log (cf. [SUPERVISION_LOGS.md](#)).
- **Pagination obligatoire** des listes (évite les gros jeux de résultats).
- **Images WebP** + mise en cache statique des assets.
- **OPcache PHP** activé en production (compilation bytecode).
- Montée en charge horizontale possible (application sans état hors session, session externalisable en Redis) — DCT §7.2.

Références

- Cahier de recettes (tests unitaires & fonctionnels)
- Supervision & logs
- DAT — Dossier d'Architecture Technique